

The AAMT Platform Architecture: Construction, Runtime, Ecosystem Integration, and Comparative Position

Authors: Weslyn Cory Whitehead Jr.¹

Affiliations: ¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: July 2026

Working Paper Series: AAMT-WP-24

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Status: Working paper, not peer reviewed. This paper is a synthesis, not a new experiment: it describes what the preceding WP series (WP-15 through WP-23, EXP-01) and their reference implementations actually build, how the resulting system runs, where it is wired into the rest of the AsAManThinks platform today, and — the part usually left implicit — precisely how that differs from the architectures it sits beside. Every claim below cites a real file, a real measured number, or an explicit statement that the claim is aspirational and not yet built. Where an earlier paper’s framing turned out to be imprecise once it met a real integration, this paper states the correction rather than the original claim (see §7).

Abstract

The AAMT stack is usually described one paper at a time — a lattice here, a substrate there, an escape-steering rule somewhere else — and the throughline is easy to lose. This paper states it directly: AsAManThinks runs a **geometric memory layer** *beside* conventional language-model inference, not instead of it. A 4-dimensional coordinate space (TERA), an exact 16-orthant tessellation of it (Meji) with a further 256-cell refinement (Odu), and a small number of provably deterministic operators (a chiral tie-break O_x , a product Fisher–Rao metric, a Householder reflection) form a mathematical layer that is deliberately independent of any model weights. A memory-mapped binary format (`.aamt`, WP-23) makes that geometry a zero-parse, cross-platform artifact — the same bytes open identically from a native CLI, from Python via `ctypes`, from a browser via WebAssembly, and from Unity via P/Invoke, with byte-identical routing verified across all four. A small, orthogonal set of runtime mechanisms sits on top of the geometry: coarse-then-fine retrieval (WP-20), rolling output eviction for unbounded generation (WP-21), session-level entropy monitoring with cross-session escape steering (WP-22, WP-23 §9), and a geometric redundancy scheme built from a single exact reflection rather than a generic checksum (WP-23 §10). We describe how this threads into six real, running consumers across the platform — the desktop training/inference stack, the DreamOS Unity client, the public math visualizer, the Oracle divination app, the InnerSpeech translation engine, and the marketing site’s production deployment — and we state plainly where the architecture’s claims are measured and where they are still aspirational. We then compare the resulting shape against three architectures it is easy to conflate it with — token-sequence-only transformer stacks, conventional vector databases, and generic RAG pipelines — and show the difference is not cosmetic: it is a difference in what the system is willing to claim it knows without running a model at all.

1. What AAMT actually is

Strip the naming and the claim is simple: **most of what a generative AI system needs to know about *where it is* in meaning-space can be answered geometrically, in constant or logarithmic time, without touching a model’s weights.** “Where is this conversation heading,” “have I said this before,” “which stored memory is nearest,” “is this a coherent reflection of what I wrote a moment ago” — these are geometry questions, not generation questions, and AAMT’s bet is that answering them geometrically is cheaper, more auditable, and more portable than asking a transformer to hold the answer implicitly in its residual stream.

The architecture that follows from that bet has three layers, built in this order and still separable in the code today:

1. **A coordinate system** (TERA) and its exact tessellation (Meji, Odu) — pure mathematics, no I/O, no model dependency (`packages/vortex-runtime`, `packages/aamt-foundations`).
2. **A persistence format** for that geometry (`.aamt`, `packages/libaamt`) that is memory-mapped, language-agnostic, and immutable once packed.
3. **A small set of runtime mechanisms** — retrieval, generation eviction, coverage monitoring, redundancy — expressed as operations *on* that geometry, each independently measured (WP-20 through WP-23).

Nothing above requires a specific model, a specific app, or a specific platform. That independence is the architectural claim this paper is actually about, and §6 shows it holding across four unrelated runtimes.

2. The mathematical layer

TERA. Every point of interest — a chunk of generated text, a memory, a query, a moment in a conversation — is projected to a 4-tuple $v = (T, E, R, A) \in [0, 1]^4$ (Temporal, Emotional, Rational, Archetypal). The projection itself is pluggable: a torch-free lexicon projector for prototyping, a trained linear head (`VortexGate.tera_proj`) for production (`apps/desktop/scripts/extract_tera_proj.py`), or the axis-major quantizer `voxel.tera_to_odu` used by the session-coverage code. All three are required to agree with each other where they overlap, and that agreement is tested, not assumed (WP-23 §9.1’s 20,000-coordinate cross-check between the substrate’s own Meji-major cell index and `voxel.py`’s axis-major one).

Meji and Odu are not decoration. A binary split of $[0, 1]^4$ produces exactly $2^4 = 16$ orthants — this is arithmetic, not a naming choice — and those 16 orthants are the 16 Meji. Two levels of splitting produce exactly $16 \times 16 = 256$ cells — the 256 Odu. The mythic names (drawn from Ifá divination’s own binary/tetragram structure) are not layered on top of a generic quadtree-in-4D; the correspondence is exact, and it is why a substrate routing path is literally readable as an Odu word (5.6.13.1) and why the Oracle app’s 256 divination cards (`apps/oracle/lib/odu-data.ts`) and the substrate’s own 256-cell tree depth-2 layer are *the same lattice*, verified bit-for-bit in §6.

$p(M_k)$ — **the soft distribution hard routing is the limit of.** For a coordinate v , the probability that it “belongs” to Meji orthant k under a product-Bernoulli model is $p(M_k) = \prod_{d \in \{T, E, R, A\}} v_d^{b_d(k)} (1 - v_d)^{1 - b_d(k)}$ (`packages/vortex-runtime/src/index.ts`, `mejiDistribution`). Hard routing (`mejiIndex` in the substrate) is exactly $\arg \max_k p(M_k)$ at the orthant’s own midpoint; soft routing (WP-23 §7, `routeSoftTopK`) is the same distribution queried directly, ranking all 16 orthants instead of picking one.

The Fisher–Rao metric gives this space a real distance, not just a partition: for product-Bernoulli coordinates, $d_{FR}(u, v) = 2 \sum_d |\arcsin \sqrt{u_d} - \arcsin \sqrt{v_d}|$ — eight `arcsin` calls, nothing

exotic, and it is the geodesic WP-22’s escape steering actually walks (§4).

O_x — the one deterministic tie-break, reused everywhere. Wherever this framework needs to resolve a genuine tie (a coordinate landing exactly on a splitting hyperplane, a local restoring force vanishing exactly at a lattice site), it uses the same rule: resolve right-continuously, deterministically, with a fixed handedness — the same object as IEEE-754 signed zero. This is not three unrelated hacks; it is one operator applied in three places that turned out to need it: the substrate’s router (`format.hpp::mejiIndex, v >= mid`), EXP-01’s chiral lattice relaxation (the O_x perturbation term), and — a genuinely separate application in a completely different app — `packages/innerspeech-core`’s translation pipeline, which uses “ O_x ambiguity detection” to decide when a translation choice is a genuine tie rather than force a false one. Three domains, one operator, because the operator’s job (name the smallest asymmetry that breaks a real tie, consistently) is domain-independent.

3. The memory/substrate layer

WP-23 specifies and `packages/libaamt` implements the persistence format: a memory-mapped hexadeca-tree over TERA space.

Format. [128B header][node arena, 64B cache-line-aligned nodes][payload arena, page-aligned]. A node’s 16 potential Meji children are stored via an occupancy bitmask + `popcount`-indexed contiguous slots — sparse regions cost zero bytes, and the struct is `static_assert`-verified to fit one cache line (`packages/libaamt/src/format.hpp`).

Ingestion is `mmap`, full stop. Opening a substrate costs one `mmap` syscall and a header check — independent of file size. Measured: `aamt_open` on a 1,000,000-record file — **0.021 ms**. The natural comparison, parsing the same data as a flat TSV/JSON store (the pre-AAMT persistence path), grows linearly with N : **64.8 ms** at 100,000 records, unbounded upward (`packages/libaamt/examples/bench_cold_start.py`).

Routing is $O(\text{depth})$, not $O(1)$ — a corrected claim from an earlier draft; the honest statement is *scan-free and bounded*, independent of how much data is stored beyond its logarithm. Measured on a 1,000,000 record substrate: **397 ns/query, 2.52M queries/s single-threaded**, including full C-ABI marshalling.

Zero-copy on Apple Silicon. Unified memory means the CPU’s routing result — a raw payload pointer — hands directly to a Metal compute shader via `newBufferWithBytesNoCopy`, no transfer. Measured (`packages/libaamt/metal`): a **4.18× speedup** over the copying alternative at 528 MB, but, stated honestly because it is real and useful data, a **0.56× loss** (copy wins) at 4.8 MB — zero-copy’s fixed setup cost is not always worth paying, and the crossover is measured, not assumed.

Deterministic replay. A sidecar `.aamtj` journal logs (`timestamp, file_hash, tera, path, depth`) — 40 bytes per routing decision — and `aamt ffi.verify_journal()` replays accumulated real history against the current substrate, not a synthetic batch, checking that the file’s content hash still matches and that every logged routing decision still reproduces bit-for-bit. This operationalizes what would otherwise be an unverified claim: that a `.aamt` file’s routing behavior is a pure function of its bytes.

Geometric redundancy. WP-23 §10 defines a Householder reflection of the TERA cube’s T -axis, $H(v) = (1 - T, E, R, A)$ — an exact involution ($H(H(v)) = v$, no drift) and a true reflection ($\det = -1$; flipping all four axes instead would be orientation-*preserving* in this even dimension, a rotation, not a mirror, despite reading like one). A mirrored record gets a full physical copy at the leaf its reflection routes to — a different subtree, a different region of the file. The gate, `verifyChirality`, re-derives every primary’s reflection for real and flags disagreement; the recovery

path, `recoverRecord`, is scoped to admit — not hide — that a 2-copy scheme with no independent checksum cannot always decide *which* copy is corrupt on a pure payload mismatch, only that they disagree. Both paths are verified against real, injected byte-level corruption (a separate file handle editing packed bytes at a computed offset), not simulated failure.

4. The runtime mechanisms

Four operations run *on* the geometry above, each independently measured and each answering a different question:

- **Coarse-then-fine retrieval (WP-20).** TERA routing alone measured $\text{recall@10} = 0.017$ — read correctly not as a weakness but as evidence TERA is the wrong tool for exact retrieval; an HNSW residual over embeddings supplies the missing precision ($\text{recall@10} = 0.331$). The substrate’s `route/query` split (§6) is this division of labor as two wire commands instead of one being quietly worse than the other.
- **Rolling output eviction (WP-21).** Generation is not held in an ever-growing KV cache; chunks are evicted into the geometric index as they are produced, queried back by coordinate rather than replayed by token position, giving effectively unbounded generation length at bounded attention cost.
- **Session coverage + escape steering (WP-22, revised in WP-23 §9).** A 256-cell occupancy count is session-local, exact, and cheap (1 KB, $O(1)$ per chunk); when a cell saturates, a Fisher–Rao-nearest cold cell becomes the escape target. WP-23 §9 closed a real gap here: escape targets for cells unvisited *this session* now fall back to the substrate’s cross-session persisted centroid (`aamt_odu_profile`) before resorting to an abstract geometric cell center — verified with seeded, unseeded, and distance controls, and occupancy counts are deliberately never seeded from the substrate, because coverage is a claim about *this session’s* trajectory and a fudged prior would corrupt the exact signal WP-22 exists to compute.
- **Geometric redundancy (WP-23 §10).** Described above; the only mechanism in this list whose job is data integrity rather than retrieval or generation.

5. How it runs

The execution model is a thin native core underneath whatever expression layer a given surface already uses — a Dual-Stack shape, but with the “stack” chosen per-consumer rather than mandated: Python where the desktop training stack already runs Python, JavaScript/WASM where the browser already runs JavaScript, C# where Unity already runs C#. A single C ABI (`packages/libaamt/include/aamt/aamt.h`) is the one boundary every layer calls; none of them re-implement the router.

Hardware use follows the asymmetry that is actually true of consumer silicon: CPUs are built for exactly the branchy pointer-chasing a tree descent is, and GPUs are built for exactly the uniform parallel arithmetic payload transforms are — and are bad at the tree descent (warp divergence on every branch). The substrate’s job split is therefore CPU-navigates, GPU/SIMD-computes, made literal on Apple Silicon by the zero-copy handoff in §3.

6. Ecosystem integration — six real consumers

This is the section that makes “cross-platform” a checked claim rather than an assertion: the same `.aamt` file, byte-identical, opens correctly from every one of these.

Consumer	What it does with AAMT	Verified
Desktop training/inference	HNSW cross-session memory index (<code>memory_index_bridge.py</code>); .aamt substrate provides <code>route/route(soft)</code> coarse lookup and <code>profile</code> (cross-session escape prior) alongside it. First production integration.	Real subprocess wire protocol, corruption-injection tests, <code>test_escape_seed.py</code> end-to-end.
WP-21 rolling generation loop	Evicted-chunk TERA coordinates feed the coverage map (<code>wp21_rolling_output_prototype.py</code>); WP21_SUBSTRATE_SEED opts into cross-session escape steering.	<code>send_request(index_proc, "profile", ...)</code> wired and logged.
DreamOS / Unity client	<code>AamtSubstrateService</code> (MonoBehaviour singleton) opens a .aamt from StreamingAssets at boot; <code>AamtSubstrateBridge</code> exposes routing to OneJS/Preact UI pages, mirroring the existing <code>VortexService/VortexBridge</code> pattern.	P/Invoke over the same native <code>libaamt.dylib</code> ; struct layouts hand-verified against the C ABI.
AAMT Math Visualizer (public)	“AAMT Substrate” tab loads a real .aamt packing the same 256 Oracle cards Oracle itself serves, routes live TERA-slider queries via the WASM build of the identical C++ core.	Content hash shown in-browser matches the native CLI’s <code>aamt inspect</code> output on the identical file, verified live.
Oracle	256 divination cards (<code>odu-data.ts</code>) sourced from <code>aamt-foundations’</code> <code>oracle-256-cards.json</code> — the same 256-cell Odu lattice the substrate routes against, same source JSON the visualizer’s demo substrate is packed from.	Shared source file; the visualizer’s low-corner/high-corner routing test lands on cards 0-0/15-15 exactly as named.

Consumer	What it does with AAMT	Verified
InnerSpeech	Reuses the O_x operator, independently, as “Ox ambiguity detection” in a translation pipeline — a different domain (language translation vs. geometric routing) applying the same deterministic-tie-break primitive.	Declared in the package’s own description; a genuine second application, not a rename.

Full paths, for the rows above in order:

- `maaiam-chemist/apps/desktop/scripts/` (first two rows)
- `MaiiaM Platform/Assets/MaiiaM/Systems/Substrate` (Unity client)
- `asamanthinks.com/chemist/visualizer` (the visualizer)
- `apps/oracle` (Oracle)
- `packages/innerspeech-core` (InnerSpeech)

Additional real dependents not detailed above: `@aamt/netergaiiam` depends directly on `aamt-foundations`; the marketing site’s production deployment (§ below) is the vehicle for the visualizer’s public-facing consumer entry.

Production deployment is real, not staged. The visualizer’s WASM build is live on `asamanthinks.com`, served from the same Next.js app that handles the platform’s marketing site (`apps/marketing`, PM2-managed on the production VPS). One real production bug shipped and was caught within minutes by live browser testing rather than by the automated suite: Emscripten’s growable WebAssembly memory produces a “resizable” `ArrayBuffer`, which Safari’s `TextDecoder` rejects outright — including inside Emscripten’s own generated glue, not just hand-written code. The fix (fixed 128 MB memory, growth disabled) eliminates the whole bug class rather than patching call sites, and `HEAPU8.buffer.resizable === false` is now asserted in the WASM test suite. Node’s V8 never exhibited the bug at all — the lesson recorded plainly rather than smoothed over: an automated suite that runs under Node cannot, by construction, catch a Safari-specific failure.

7. What is honestly still incomplete

In the tradition the EXP-01/WP-23 series has already established — record what was wrong, not just what is right:

- **Unity/C# bindings were written without a compiler available in-session** and are hand-verified against the C ABI, not yet build-confirmed by the Unity Editor at the time of writing (a background verification task was in flight; `.meta` files generated by opening the project have since appeared, a good sign, not yet a substitute for a green build log).
- **Windows and Linux native builds do not exist.** `libaamt.dylib` is arm64-only, built on Apple Silicon; a shipped cross-platform build needs its own `build.sh` run per target, not yet done.
- **The 2-copy redundancy scheme has a real, stated ceiling**, not a bug: it cannot always decide which of two disagreeing copies is corrupt without a third copy or an independent checksum, neither of which the format stores. This is demonstrated by a passing test that asserts the *wrong* value comes back in that specific case, on purpose.

- **The columnar bulk-export path (WP-23 corrections ledger #8) is $2.84\times$ faster than the naive per-record loop but still loses to `json.load` in Python specifically** — CPython’s C-implemented JSON parser has no per-element bytecode loop at all; the columnar path is kept because Unity and the WASM build have no such Python-specific competitor to lose to, not because it “solved” the Python case.
- **@aamt/vortex-runtime’s TypeScript package graph currently shows zero declared consumers** in the workspace’s own `package.json` dependency edges, despite the Unity `VortexService/VortexBridge` classes citing it as their explicit source of truth — the math was ported by hand rather than imported. Whether that is a packaging gap worth closing or an acceptable one-way port is an open question, not resolved here.

8. How this differs from other architectures

Versus token-sequence-only transformer stacks. A standard LLM-serving stack has exactly one representation of “where the conversation is”: whatever is implicit in the current KV cache and the model’s residual stream. There is no cheap, inspectable, model-independent answer to “is this drifting” or “have I been here before” — you either re-run the model to find out, or you don’t ask. AAMT’s bet is that a large fraction of that question is geometric and answerable in nanoseconds without a forward pass — WP-22’s $O(1)$ -per-chunk coverage check and WP-23’s 397 ns routing are that bet, measured.

Versus conventional vector databases. Systems built around a single ANN index (HNSW, IVF, a flat index) optimize one thing: nearest-neighbor recall over a fixed embedding space. AAMT deliberately runs two indices with different, measured jobs — TERA’s hexadeca-tree for coarse, geometric, $O(\text{depth})$ structure and an HNSW residual for fine discrimination — because WP-20 measured that a single coarse index cannot do both (recall@10 = 0.017 alone) and that trying to make it do both is the wrong fix. A conventional vector DB has no equivalent of “coarse questions get a cheap, separate, correct answer instead of a degraded version of the precise one.”

Versus generic RAG pipelines. A typical RAG system’s persistence is server-side, schema-specific, and single-consumer: an application queries its own vector store over its own network boundary. `.aamt` is a single portable file, `mmap`-opened identically by a CLI, a Python process, a browser tab, and a Unity client, with the identity of “the same file” checked by content hash and cross-verified by routing behavior, not asserted. Deterministic replay (the lineage journal) and a persisted-profile handoff between independent sessions (WP-23 §9) are both artifacts of treating the memory as a portable, self-describing object rather than a private index behind one application’s API.

Versus generic redundancy/error-correction. RAID-style mirroring and checksums are domain-agnostic — any bytes, any layout. AAMT’s redundancy (§3, §WP-23 §10) is built from an operation *native to the domain’s own geometry* — a reflection of the same coordinate space everything else in the system already routes through — so the “mirror” is not bolted onto the format, it is a second, cheap use of the format’s own math. The honest limitation stated in §7 (2 copies cannot always decide which is corrupt) is also the same limitation any 2-way mirror has; AAMT does not claim otherwise, which is itself a difference from architectures that market redundancy without stating its ceiling.

Versus opaque or borrowed terminology. The Meji/Odu naming is not a skin over a generic quadtree, and stating that precisely is itself a point of difference: most systems that borrow evocative names for technical structures (and many do) are decorating an implementation with a metaphor. Here the correspondence is exact and load-bearing — the orthant count *is* 16 because the split is 4-dimensional binary, not because 16 was chosen and then named. A reader who doubts

this can check it directly: query the visualizer’s demo substrate at the TERA cube’s exact corners and the routing path names the identically-numbered oracle card, live, in a browser (§6).

9. Conclusion

Nothing in this paper is a new experiment. It is an accounting: a mathematical layer with three provably reused operators, a persistence format whose cross-platform identity is checked rather than assumed, four runtime mechanisms each measured against a stated, falsifiable claim, six real running consumers spanning a training stack, a game engine, a browser, a divination app, a translation engine, and a public production deployment, and — the part that matters for trusting any of the above — an explicit list of what is not yet true. The architecture’s actual claim to being different from what came before is not the lattice or the naming; it is that the system will tell you, in the same document, both what it measured and what it got wrong.

Reference implementation: `packages/libaamt`, `packages/vortex-runtime`, `packages/aamt-foundations`.
Consumers cited: `maiam-alchemist/apps/desktop`, `Maiam Platform/Assets/Maiam/Systems/Substrate`,
the AAMT Mathematics Visualizer, `apps/oracle`, `packages/innerspeech-core`, `apps/marketing`.
Prior series: WP-15 (Fisher–Rao geometry), WP-17 (factorized entropy), WP-20 (voxel-addressable memory), WP-21 (voxel-steered generation), WP-22 (Odu coverage map), WP-23 (the `.aamt` substrate), EXP-01 (topological trit memory).